

P9 Report

Time Complexity

Let n be the number of followers stored for the specified user. Since usernames have a maximum length of 50 characters, so could treat the length of each username as a constant.

The deserialise operation scans the stored follower string and reconstructs the follower array, taking $O(n)$ time. The function then searches up to n followers to determine whether the user has already existed in the array. If the follower is new, the serialise operation processes the follower list again, taking $O(n)$ time.

Above all, ignoring the hash collisions, the whole complexity should be:

$O(n)$.

However, if there are multiple keys points the same position in the hash table, we still need to take $O(c)$ time, which c is the number of entries in that hash-table chain, to find the result needed in the worst case. Therefore, the worst case is $O(n+c)$ and the best case is $O(1)$ when the key is at the first node and the value of corresponding key is empty.

Evidence of Testing

```
FollowersDatabase db;
db.ht.insert(key: "A", value: "");
db.ht.insert(key: "B", value: "");

//Test add_follower()
cout<< "Test add_follower()"<<endl;
cout<< "Add A1 for A:" << endl;
db.add_follower( username: "A", follower: "A1");
db.print_followers( username: "A");

cout << "\nAdd B1, B2, B3 for B:"<<endl;
db.add_follower( username: "B", follower: "B1");
db.add_follower( username: "B", follower: "B2");
db.add_follower( username: "B", follower: "B3");
db.print_followers( username: "B");

//Test delete_follower()
cout<< "\nTest delete_follower()"<<endl;
cout<< "Delete B2 for B:" << endl;
db.delete_follower( username: "B", follower: "B2");
db.print_followers( username: "B");
```

```
/Users/yihaoyu/Desktop/FIT2085/Ontrack_Files/P9/main
```

```
Test add_follower()
```

```
Add A1 for A:
```

```
A1
```

```
Add B1, B2, B3 for B:
```

```
B1
```

```
B2
```

```
B3
```

```
Test delete_follower()
```

```
Delete B2 for B:
```

```
B1
```

```
B3
```